



Ministerul Educației și Cercetării
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

Titlul extins al lucrării de absolvire

Lucrare de licență

Coordonator științific:

Lect. univ. dr. Rusu Andrei

Absolvent:

Ene Luis Adrian

Constanța
2026

Cuprins

Cuprins	i
Lista Figurilor	vi
Rezumat	2
Rezumat	2
Abstract	3
Abstract	3
1 Motivație	4
1.1 Contextul si problema	4
1.2 Obiectivele lucrării	5
1.3 Structura documentului	5
1.4 Tipuri de evenimente vizate	6
1.4.1 Conferinte, meeting-uri si evenimente profesionale	6
1.4.2 Concerte si evenimente culturale	6
1.4.3 Evenimente sportive si utilizare complementara	7
1.4.4 Beneficii transversale	7

2	Starea actuală a domeniului	8
2.1	Studii și articole științifice	8
2.1.1	Arhitecturi web și API-uri REST	8
2.1.2	Securitatea autentificării	8
2.1.3	Plăți online și gateway-uri	9
2.1.4	Persistență și baze de date relaționale	9
2.1.5	Interfețe utilizator și SPA	9
2.2	Aplicații dedicate domeniului	9
2.2.1	Eventbrite	9
2.2.2	Platforme locale și generice	10
2.2.3	Tendințe și standarde în industria ticketing-ului	10
2.2.4	Conformitate PCI-DSS și delegarea plăților	11
2.2.5	Arhitectura SPA și API REST	11
2.2.6	Studiu de caz: fluxul utilizatorului în platforme consacrate	11
2.3	Criterii de evaluare a soluțiilor	12
2.4	Analiză comparativă	12
2.5	Concluzii parțiale	13
3	Soluția propusă	14
3.1	Cerinte functionale pentru utilizator si nefunctionale	14
3.1.1	Cerinte funtionale pentru utilizator	14
3.1.2	Cerințe funcționale pentru administrator	14
3.1.3	Cerințe nefuncționale	15
3.2	Arhitectura sistemului	15
3.3	Tehnologii utilizate	16
3.4	Diagrame UML	16
3.4.1	Cazuri de utilizare	16
3.4.2	Secventa de inregistrare cu verificare prin email	17

3.4.3	Secventa pentru plata prin Stripe	17
3.4.4	Activitatea la check-in si admin	18
3.4.5	Proiectarea bazei de date	18
3.5	Proiectarea bazei de date	18
3.6	Implementarea functionalitatilor cheie	20
3.6.1	Autentificarea si verificarea prin email	20
3.6.2	Plati prin Stripe	21
3.6.3	Gestionare evenimente	21
3.6.4	Frontend	21
3.7	Securitate	22
3.8	Scenarii de utilizare pe tipuri de evenimente	22
3.8.1	Scenariul 1: Conferinta universitara	22
3.8.2	Scenariul 2 in care concertul este in aer liber	23
3.8.3	Scenariul 3: Meeting corporate / training	23
3.8.4	Scenariul 4: Competitie sportiva amatoriala	23
3.8.5	M	23
3.8.6	Modelul unificat de date	23
3.9	Fragmente suplimentare de implementare	24
3.9.1	Rate limiting la inregistrare	24
3.9.2	Middleware autentificare JWT	25
3.9.3	Validare evenimente la creare	25
3.9.4	Componenta AdminRoute (frontend)	26
3.9.5	Gestionare erori si feedback utilizator	26
3.10	Modelul relational - detaliu tavele	26
3.10.1	Tabel user	26
3.10.2	Tabele auxiliare	27
3.11	API REST - referinta	27

3.12	Configurare si deployment	28
3.12.1	Variabile de mdio backend	28
3.12.2	Migrari baza de date	28
3.12.3	Flux instalare	28
3.12.4	Componente frontend	28
3.12.5	Confirmare plată — pseudocod	29
3.12.6	Testare manuală	29
4	Prezentarea aplicației	30
4.1	Utilizarea aplicației — perspectiva utilizatorului	30
4.1.1	Pagina principală	30
4.1.2	Detalii eveniment	30
4.1.3	Înregistrare cont — pasul 1	32
4.1.4	Verificare email — pasul 2	32
4.1.5	Autentificare	32
4.1.6	Confirmare plată	32
4.1.7	Biletele mele	34
4.1.8	Profil utilizator	34
4.1.9	Resetare parolă — pasul 1 (optional)	35
4.1.10	Resetare parolă — pasul 2 (optional)	36
4.2	Experiență utilizatorului și design responsive	36
4.2.1	Parcursul utilizatorului (user journey)	36
4.2.2	Adaptare mobil și accesibilitate	37
4.2.3	Mesaje de eroare și stări goale	37
4.3	Utilizarea aplicației — perspectiva administratorului	37
4.3.1	Panou admin — dashboard	37
4.3.2	Creare / editare eveniment	37
4.3.3	Check-in participant	39

4.3.4	Gestionare administratori	39
4.3.5	Conferință sau simpozion	40
4.3.6	Concert sau spectacol	40
4.3.7	Meeting, workshop sau seminar	41
4.3.8	Eveniment sportiv (caz secundar)	41
4.3.9	Recomandări pentru capturi de ecran (capitol 4)	41
4.3.10	Accesibilitate și public larg	41
5	Concluzii	42
5.1	Rezultate obținute	42
5.2	Limitări	42
5.3	Dezvoltări viitoare	43
	Referințe bibliografice	44

Lista Figurilor

3.1	Arhitectura sistemului ManFast	15
3.2	Diagrama cazurilor de utilizare	16
3.3	Diagrama de secventa - inregistrarea cu email	17
3.4	Diagrama de secventa - plata Stripe	18
3.5	Diagrama de activitate - check-in	19
3.6	Diagrama entitate- relatie	19
3.7	Diagrama entitate- relatie	20
4.1	Pagina principală — listă evenimente și filtre	31
4.2	Pagina detalii eveniment și formular bilete	31
4.3	Formular inregistrare	32
4.4	Verificare email cu cod	33
4.5	Pagina login	33
4.6	Pagina succesc plata	34
4.7	Biletele mele	34
4.8	Profil utilizator	35
4.9	Pagina Am uitat parola (opt_ional)	35
4.10	Formular parola noua (reset)	36
4.11	Panou admin — dashboard	38

4.12	Dialog editare eveniment	38
4.13	Check-in participant	39
4.14	Check-in participant validat	39
4.15	Gestionare administratori	40

Rezumat

Lucrarea de licență *Aplicație web pentru organizarea și gestionarea evenimentelor* propune dezvoltarea unei platforme complete destinate organizatorilor și participanților la conferințe, meeting-uri, concerte, evenimente culturale și, într-o măsură mai mică, competiții sportive. Aplicația (implementare: proiectul ManFast) permite publicarea evenimentelor, filtrarea după nume, județ și dată, achiziționarea biletelor nominale cu plată online prin Stripe și administrarea participanților printr-un panou dedicat.

Soluția tehnică adoptă arhitectura client-server: frontend React 19 cu Material UI 9, backend Node.js/Express, bază de date PostgreSQL. Autentificarea utilizează token-uri JWT, parole hash-uite cu bcrypt, verificare email la înregistrare (cod de 6 cifre) și resetare parolă. Administratorii gestionează evenimente (inclusiv imagini), efectuează check-in și promovează alți administratori.

Documentul prezintă motivația temei, starea domeniului, proiectarea și implementarea soluției, capturi din aplicația funcțională și concluzii. Platforma reprezintă o soluție scalabilă, adaptabilă contextului românesc (plăți RON, județe).

Abstract

The bachelor's thesis Web application for organizing and managing events proposes the development of a complete platform for organizers and participants in conferences, meetings, concerts, cultural events, and to a lesser extent sports competitions. The application (implementation: ManFast project) enables event publishing, filtering by name, county and date, purchasing nominal tickets with online payment via Stripe, and participant management through a dedicated admin panel.

The technical solution adopts a client-server architecture: React 19 frontend with Material UI 9, Node.js/Express backend, PostgreSQL database. Authentication uses JWT tokens, bcrypt password hashing, email verification on registration (6-digit code), and password reset. Administrators manage events (including images), perform check-in, and promote other administrators.

The document presents the motivation, domain state, solution design and implementation, application screenshots, and conclusions. The platform represents a scalable solution adapted to the Romanian context (RON payments, counties).

Capitolul 1

Motivație

1.1 Contextul si problema

Organizarea **conferințelor, meeting-urilor, concertelor** și a altor evenimente cu public și locuri limitate implică, în mod tradițional, procese fragmentate: promovarea prin canale disparate (rețele sociale, email, site-uri proprii), vânzarea biletelor sau înscrierilor în format fizic sau prin transfer bancar manual, și validarea participanților la intrare prin liste pe hârtie. Această abordare generează erori, pierderi de timp și dificultăți în reconcilierea plăților cu locurile rezervate.

Digitalizarea procesului — centralizarea informațiilor despre evenimente, automatizarea plăților și asocierea fiecărui bilet cu identitatea participantului — reprezintă un pas natural în contextul extinderii serviciilor online și al așteptărilor utilizatorilor privind comoditatea și securitatea tranzacțiilor [1, 2].

Organizatorii (universități, firme, asociații culturale, săli de concert) nu dispun adesea de resurse pentru soluții enterprise costisitoare. În același timp, platformele internaționale generice nu sunt întotdeauna adaptate pieței locale: moneda RON, structura administrativă pe județe, sau necesitatea unui flux simplu de check-in administrat de organizator. Pentru **evenimente sportive** amatoriale, aceleași mecanisme (înscriere, plată, listă nominală) sunt utile, dar reprezintă un caz secundar față de conferințelor și concertelor.

Participanții au nevoie de o interfață clară: descoperire evenimente, filtrare, achiziție rapidă unuia sau mai multor bilete nominale, vizualizarea istoricului și gestionarea contului. Organizatorii au nevoie de un panou unic pentru publicare, monitorizare locuri disponibile și confirmare prezență la fața locului.

1.2 Obiectivele lucrării

Obiectivul principal este proiectarea și implementarea **aplicației web pentru organizarea și gestionarea evenimentelor**, care să îndeplinească următoarele obiective specifice:

1. Implementarea unui modul de autentificare securizat: înregistrare cu verificare email, login JWT, resetare parolă, editare profil și ștergere cont.
2. Oferirea unei interfețe publice pentru listarea evenimentelor viitoare, cu filtre după nume, județ și dată.
3. Permitea achiziționării biletelor nominale (unul sau mai multe nume per tranzacție) cu plată online prin Stripe Checkout, în moneda RON.
4. Dezvoltarea unui panou de administrare pentru CRUD evenimente (inclusiv imagini), listă participanți și check-in.
5. Gestionarea rolurilor utilizator/admin și promovarea/revocarea administratorilor cu parolă de autorizare.
6. Documentarea arhitecturii, a modelului de date și a fluxurilor principale (UML, ERD).

Lucrarea demonstrează integrarea tehnologiilor web moderne într-un produs coerent, testabil și prezentabil, cu respectarea unor practici de securitate recunoscute [?, 3].

1.3 Structura documentului

Prezenta lucrare este organizată structural pe capitole, după cum urmează:

Capitolul 2 Analizează în detaliu starea actuală a domeniului. Cuprinde studii relevante din literatură, o analiză a platformelor existente pe piață și o comparație directă cu soluția propusă.

Capitolul 3 Descrie în profunzime arhitectura sistemului și detaliile de implementare ale aplicației. Sunt incluse cerințele funcționale și non-funcționale, tehnologiile alese, diagramele UML, structura bazei de date și fragmente de cod sursă relevante.

Capitolul 4 Prezintă interfețele grafice ale aplicației prin intermediul unor capturi de ecran detaliate, mapate pe fluxurile principale de utilizare.

Capitolul 5 Sintetizează concluziile generale ale proiectului, evidențiază limitările identificate pe parcursul dezvoltării și propune viitoarele direcții de dezvoltare.

1.4 Tipuri de evenimente vizate

Aplicația web pentru organizarea și gestionarea evenimentelor este concepută în jurul unor formate frecvente în viața academică, culturală și profesională. Tabelul 1.1 sintetizează categoriile principale, actorii implicați și modul în care platforma le susține.

Tip Eveniment	Actori Principali	Scenariu în Aplicație
Conferință / Meeting	Organizator, Participant	Publicare cu detalii complete (dată, locuri limitate, descriere, preț în RON), înregistrare online și check-in la intrare.
Concert / Festival	Organizator, Public	Accent pe componenta vizuală (galerie de imagini), redirectionare rapidă către Stripe Checkout pentru achiziție bilet nominal.
Competiție Sportivă	Admin, Sportiv	Înscriere pe bază de bilet nominal, check-in realizat de organizator la înregistrarea în concurs (caz secundar).

Tabelul 1.1: Tipuri de evenimente reflectate în aplicație

1.4.1 Conferinte, meeting-uri si evenimente profesionale

Universități, camere de comerț și firme de consultanță organizează periodic conferințe și meeting-uri cu număr limitat de locuri. Participanții trebuie să se înregistreze, să-și confirme identitatea (email verificat) și, uneori, să achite o taxă de participare. Platforma acoperă acest flux end-to-end: publicarea evenimentului cu dată, locație (județ, localitate) și capacitate, urmată de înregistrare online și emitere bilet nominal.

Pentru meeting-uri interne sau seminarii plătite, administratorul poate crea evenimente cu preț fix și număr redus de locuri; sistemul decrementează automat locurile disponibile la fiecare tranzacție confirmată. Check-in-ul din panoul admin permite validarea prezenței la recepție sau la intrarea în sală — util atât pentru conferințe, cât și pentru training-uri corporate.

1.4.2 Concerte si evenimente culturale

Concertele și festivalurile locale necesită o prezentare atractivă (titlu, descriere, mai multe imagini) și vânzare rapidă a biletelor. Aplicația permite încărcarea a până la opt imagini per eveniment, afișarea galeriei pe pagina de detalii și redirect către Stripe Checkout pentru plată securizată.

Filtrarea după județ și dată ajută publicul să descopere evenimente din apropiere — relevant pentru turnee naționale de concerte sau festivaluri itinerante. Biletele nominale asigură

că fiecare loc este asociat unui participant, reducând revânzarea neautorizată și simplificând controlul la intrare.

1.4.3 Evenimente sportive si utilizare complementara

Modelul actual al aplicației (nume pe bilet, check-in administrat manual) se pretează și competițiilor sportive amatoriale sau turneele locale, unde organizatorul trebuie să verifice identitatea participantului. Această nișă este secundară față de conferințe, concerte și meeting-uri: nu sunt implementate funcționalități specifice sportului (categorii de vârstă, echipe, program meciuri), dar fluxul general de înscriere și plată rămâne valabil.

Extinderi viitoare (categorii de bilete, abonamente de sezon) ar putea întări suportul sportiv fără a modifica arhitectura de bază.

1.4.4 Beneficii transversale

Indiferent de tipul evenimentului, organizatorul beneficiaza de:

- ▷ **Un singur panou** pentru toate formatele — nu este nevoie de soluții separate pentru conferințe vs. concerte.
- ▷ **Trasabilitate plăți** — fiecare bilet este legat de o sesiune Stripe și de un utilizator autentificat.
- ▷ **Adaptare locală** — județe românești, monedă RON, emailuri prin SMTP Brevo.
- ▷ **Control asupra datelor** — baza PostgreSQL self-hosted, fără dependență de un marketplace global.

Această diversitate de cazuri de utilizare justifică alegerea unui model de date simplu (eveniment unic, preț unic, bilete nominale) care rămâne suficient de expresiv pentru majoritatea scenariilor întâlnite.

Capitolul 2

Starea actuală a domeniului

În acest capitol este analizat contextul în care se încadrează aplicația web pentru organizarea și gestionarea evenimentelor: cercetări relevante pentru comerț electronic, ticketing online și securitate web, platforme existente și o comparație funcțională.

2.1 Studii și articole științifice

2.1.1 Arhitecturi web și API-uri REST

Fielding [1] definește principiile REST (Representational State Transfer) ca stil arhitectural pentru sisteme distribuite. Separarea clientului de server, utilizarea resurselor adresabile prin URI și a metodelor HTTP standard (GET, POST, PUT, DELETE) facilitează dezvoltarea aplicațiilor web scalabile. ManFast expune un API REST în Node.js/Express, consumat de frontend-ul React — abordare aliniată cu recomandările din literatura de specialitate.

Fowler [3] descrie tipare de proiectare enterprise, inclusiv separarea responsabilităților între straturi (prezentare, logică de business, persistență). Aplicația analizată respectă acest principiu: React gestionează UI, Express conține regulile de business, PostgreSQL stochează datele.

2.1.2 Securitatea autentificării

OWASP [4] enumeră bune practici pentru autentificare: hashuirea parolelor (nu stocarea în clar), protecția sesiunilor, limitarea ratei de cereri, validarea input-ului. ManFast implementează bcrypt pentru parole, JWT cu expirare 24h, rate limiting pe codurile de verificare email și resetare parolă.

Standardul JWT (JSON Web Token) [5] permite transmiterea securizată a claims-urilor

între părți. Token-ul semnat conține id-ul utilizatorului și rolul (user/admin), verificat la fiecare cerere protejată.

Provos și Mazières [6] introduc bcrypt, algoritm de hash adaptat la cost computațional configurabil, rezistent la atacuri brute-force pe parole. ManFast folosește cost factor 10.

2.1.3 Plăți online și gateway-uri

Documentația Stripe [2] descrie fluxul Checkout: sesiune de plată găzduită, redirect către pagina securizată Stripe, confirmare prin webhook sau verificare sesiune. ManFast creează sesiuni Checkout cu metadata (event_id, user_id, nume participanți) și confirmă plata prin endpoint dedicat, fără a stoca date de card pe serverul propriu — reducând responsabilitatea PCI-DSS.

2.1.4 Persistență și baze de date relaționale

PostgreSQL [7] oferă tranzații ACID, constrângeri de integritate referențială și tipuri avansate (ex.: array pentru imagini eveniment). Schema ManFast exploatează aceste capabilități pentru consistența bilete–locuri disponibile.

2.1.5 Interfețe utilizator și SPA

React [8] și ecosistemul său permit construirea de Single Page Applications reactive. Material UI [9] oferă componente accesibile și responsive. ManFast utilizează React 19, Vite pentru build rapid și MUI 9 cu temă dark personalizată.

2.2 Aplicații dedicate domeniului

2.2.1 Eventbrite

Eventbrite [10] este o platformă internațională pentru crearea și promovarea evenimentelor, cu vânzare bilete, pagini de destinație și instrumente de marketing. Oferă check-in prin cod QR și analiză audiență. Platforma suportă multiple tipuri de evenimente: concerte, conferințe, workshop-uri. Fluxul tipic: organizatorul creează pagina evenimentului, setează prețurile și capacitatea, promovează link-ul, iar participanții achiziționează bilete online.

Limitări pentru organizatori mici: comisioane per bilet, personalizare redusă a fluxului de check-in, dependență de ecosistemul proprietar. Datele participanților sunt stocate pe

infrastructura Eventbrite, nu în controlul direct al organizatorului. Pentru conferințe cu sesiuni paralele sau concerte cu locuri numerotate, unele funcționalități avansate pot lipsi sau necesita add-on-uri plățite.

Din perspectiva tehnologică, astfel de platforme folosesc arhitecturi cloud scalabile, procesare plăți prin PSP integrat și aplicații mobile pentru scanare QR. ManFast adoptă un subset al acestor capabilități, orientat spre simplitate și control local al datelor în PostgreSQL.

aici pun poza cu aplicatia

2.2.2 Platforme locale și generice

Pe piața românească există soluții pentru bilete la concerte și festivaluri (iaBilet, Eventim), precum și platforme e-commerce generice (WooCommerce, Shopify) care pot vinde “produse” reprezentând locuri la eveniment. Acestea sunt optimizate pentru vânzare la scară largă, nu neapărat pentru meeting-uri corporate sau conferințe universitare.

Organizatorii locali folosesc frecvent formulare Google, plăți directe și liste Excel — soluții cu cost zero inițial, dar cu risc ridicat de erori și lipsă de audit trail pentru plăți. Reconcilierea manuală a listei de participanți cu transferurile bancare consumă timp și poate genera dispute la intrarea în sală.

Aplicația propusă încercă să ofere un echilibru: cost de operare redus (self-hosted), flux clar de la descoperire eveniment la plată și check-in, fără complexitatea unei platforme globale. Adaptarea la județele românești și plăți RON via Stripe reprezintă un avantaj competitiv în context local față de soluțiile generice internaționale.

2.2.3 Tendințe și standarde în industria ticketing-ului

Industria biletelor electronice a crescut accelerat după pandemia COVID-19, când contactless check-in și plățile online au devenit normă. Platformele mature investesc în trei direcții: (1) experiență mobilă — aplicații native sau PWA pentru scanare rapidă; (2) analitică — dashboard-uri de vânzări, heatmap-uri de prețuri dinamice; (3) integrare marketing — email automation, remarketing pe rețele sociale.

ManFast nu urmărește replicarea tuturor acestor capabilități enterprise, ci funcționalitatea minimă viabilă pentru organizatori locali: publicare eveniment, vânzare online, listă participanți, check-in la intrare. Această decizie de scope este aliniată cu principiul MVP (Minimum Viable Product) din literatura de product management.

2.2.4 Conformitate PCI-DSS și delegarea plăților

Standardul PCI-DSS (Payment Card Industry Data Security Standard) impune măsuri stricte pentru orice sistem care procesează, stochează sau transmite date de card. Soluțiile care folosesc Stripe Checkout hosted delegă colectarea datelor sensibile către Stripe — comerciantul primește doar confirmarea plății prin API. ManFast adoptă explicit acest model: frontend-ul redirecționează utilizatorul pe domeniul checkout.stripe.com, iar backend-ul verifică statusul sesiunii fără a vedea numărul de card.

Această abordare reduce suprafața de atac și simplifică auditul de securitate pentru o lucrare de licență, menținând totodată un flux profesional de plată recunoscut de utilizatori.

2.2.5 Arhitectura SPA și API REST

Aplicațiile Single Page Application (SPA) [8] încarcă o singură pagină HTML și navighează prin JavaScript, obținând date de la server prin API JSON. Avantaje: interfață reactivă, separare clară frontend/backend, posibilitatea de a extinde ulterior cu aplicație mobilă consumând același API.

Dezavantaje relevante pentru ManFast: SEO limitat pe pagini dinamice (acceptabil pentru platformă cu autentificare), necesitatea gestionării token-ului JWT pe client. Comparția cu arhitectura server-side rendering (Next.js) ar fi o extensie viitoare; pentru licență, Vite + React oferă timp de dezvoltare redus și ecosistem bogat (Material UI).

2.2.6 Studiu de caz: fluxul utilizatorului în platforme consacrate

Pe Eventbrite, utilizatorul parcurge: descoperire eveniment → selectare tip bilet → checkout → email confirmare cu PDF/QR. Organizatorul accesează dashboard cu statistici vânzări și export CSV participanți.

ManFast simplifică acest flux pentru organizatori locali (conferințe, concerte, meeting-uri):

- un singur tip de bilet pe eveniment (preț unic);
- nume nominal obligatoriu (identificare la check-in);
- fără marketplace global — evenimentele sunt listate doar pe instanța ManFast;
- check-in manual din panou admin, fără aplicație mobilă dedicată scanării QR (extensie viitoare).

Această simplificare reduce complexitatea implementării și timpul de învățare pentru administratori neexperimentați în IT.

2.3 Criterii de evaluare a soluțiilor

Pentru a fundamenta comparația din tabelul 2.2, au fost definite criterii ponderate:

Adecvare funcțională (40%) — suport bilete nominale, check-in, filtrare locală.

Cost total de operare (20%) — comisioane PSP, abonamente platformă, hosting.

Control date (20%) — unde sunt stocate datele participanților, export, GDPR.

Personalizare (10%) — posibilitatea modificării fluxului sau branding.

Securitate autentificare (10%) — verificare email, reset parolă, roluri.

ManFast obține scor ridicat la control date și personalizare (cod deschis, PostgreSQL propriu), scor mediu la cost (Stripe per tranzacție, hosting self-managed), și scor bun la adecvare funcțională pentru nișa aleasă.

Aceste analize suplimentare contextualizează poziționarea ManFast față de soluțiile existente și justifică alegerile de scope ale proiectului.

2.4 Analiză comparativă

Tabelul 2.1 sintetizează diferențele între platforme reprezentative și aplicația propusă.

Funcționalitate	Eventbrite	Generic e-comm	Aplicația propusă
Bilete nominale	Da	Parțial	Da
Plăți Stripe RON	Nu (alte PSP)	Variabil	Da
Check-in admin	Da (QR)	Nu	Da (listă)
Verificare email register	Da	Variabil	Da (cod 6 cifre)
Filtrare județ/dată	Da	Nu	Da (42 județe)
Cod deschis / personalizabil	Nu	Nu	Da
Reset parolă tel.+email	Variabil	Variabil	Da

Tabelul 2.1: Comparație funcțională

2.5 Concluzii parțiale

Analiza demonstrează că există un spațiu pentru o soluție adaptată: conferințe, meeting-uri, concerte și evenimente culturale, cu plăți RON via Stripe, bilete pe nume și administrare simplă. Aplicația propune o implementare open-source, extensibilă, care integrează funcționalitățile esențiale și le aliniază cu cerințele locale (județe, SMTP Brevo pentru email-uri).

Următorul capitol detaliază arhitectura, tehnologiile și implementarea soluției propuse.

Capitolul 3

Soluția propusă

3.1 Cerinte functionale pentru utilizator si ne-functionale

3.1.1 Cerinte functionale pentru utilizator

- ▷ **RF-U1** — Înregistrare cont: username, email, telefon, parolă; trimitere cod 6 cifre pe email; creare cont după verificare.
- ▷ **RF-U2** — Autentificare cu email și parolă; primire token JWT.
- ▷ **RF-U3** — Resetare parolă prin email sau telefon (email mascat la căutare după telefon).
- ▷ **RF-U4** — Vizualizare listă evenimente viitoare; filtre: nume, județ, dată.
- ▷ **RF-U5** — Pagină detalii eveniment: descriere, imagini, locație, preț, locuri.
- ▷ **RF-U6** — Cumpărare bilete nominale (1..N nume); redirect Stripe Checkout.
- ▷ **RF-U7** — Confirmare plată; vizualizare bilete în “Biletele mele” (viitoare/trecute).
- ▷ **RF-U8** — Editare profil; schimbare parolă; ștergere cont (cu confirmare parolă).

3.1.2 Cerințe funcționale pentru administrator

- ▷ **RF-A1** — CRUD evenimente: titlu, descriere, dată, județ, localitate, locație, preț, locuri, durată, imagini (max 8).
- ▷ **RF-A2** — Listă participanți per eveniment; check-in.
- ▷ **RF-A3** — Promovare/revocare rol admin (cu parolă autorizare); protecție ultim admin.

3.1.3 Cerințe nefuncționale

- ▷ **RNF-1 Securitate** — bcrypt, JWT, hash token-uri/coduri, CORS, rate limiting.
- ▷ **RNF-2 Disponibilitate** — API stateless; DB relațională.
- ▷ **RNF-3 UX** — Responsive; temă dark; mesaje eroare clare.
- ▷ **RNF-4 Mentenabilitate** — Separare frontend/backend; migrări SQL.
- ▷ **RNF-5 Portabilitate** — Node.js 18+, PostgreSQL 14+.

3.2 Arhitectura sistemului

ManFast urmează arhitectura **client-server** în trei straturi:

1. **Strat prezentare** — SPA React (Vite), routing React Router, context autentificare, apeluri `fetch` către API.
2. **Strat aplicație** — Express: rute `/api/auth`, `/api/events`, `/api/tickets`; middleware JWT și `requireAdmin`.
3. **Strat date** — PostgreSQL; fișiere statice imagini în `uploads/`.

Servicii externe: **Stripe** (plăți), **Brevo SMTP** (email verificare și reset).

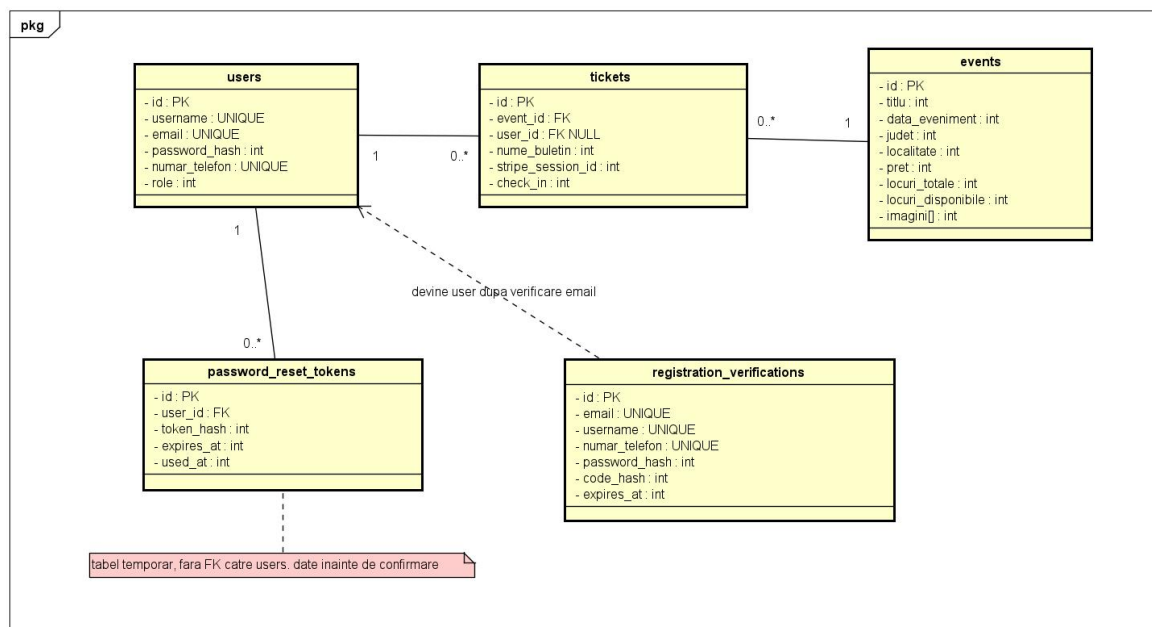


Figura 3.1: Arhitectura sistemului ManFast

Instructiuni inserare: Diagrama blocuri: Browser (React+MUI) -> HTTPS/JSON -> Express API -> PostgreSQL; lateral Stripe si Brevo SMTP. Tool: draw.io, Lucidchart sau PowerPoint. Export PNG 300dpi, latime ~14cm ~ in document.

3.3 Tehnologii utilizate

Tehnologie	Rol
React 19 + Vite 8	Frontend SPA, HMR dev
Material UI 9	Componente UI, temă dark
Express 4	API REST, Multer upload
PostgreSQL	Persistență relațională
JWT + bcrypt	Auth și hash parole
Stripe Checkout	Plăți card RON
Nodemailer + Brevo	Email SMTP

Tabelul 3.1: Stack tehnologic ManFast

React [8] gestioneaza starea UI si re-renderarea eficienta.Vite ofera build rapid. Express [11] routeaza cererile HTTP. pg conecteaza la PostgreSQL. Stripe SDK creeaza sesiuni Checkout.

3.4 Diagrame UML

3.4.1 Cazuri de utilizare

Actorii principali sunt Utilizatorul (autentificat sau nu) si Administratorul. Utilizatorul poate inregistra cont, autentifica, filtra evenimente, cumpara bilete, gestiona profilul. Administratorul publica evenimente, face check-in, gestioneaza alti admini.

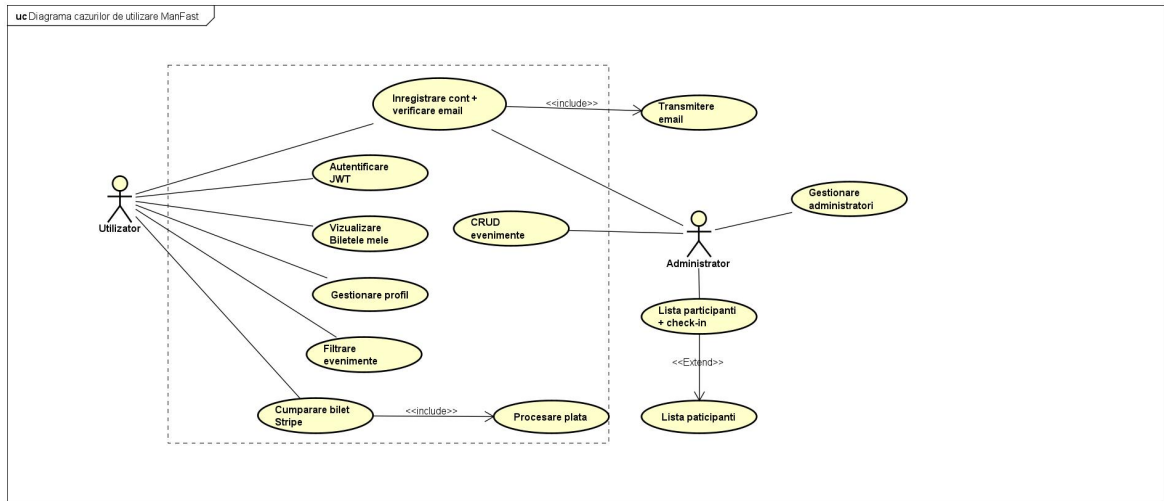


Figura 3.2: Diagrama cazurilor de utilizare

Instructiuni inserare: UML use case: actor Utilizator (register, login, filtre, cumpara

bilet, profil); actor Admin (CRUD eveniment, check-in, promovare admin). Tool: StarUML, draw.io. Include sistem boundary “ManFast”.

3.4.2 Secventa de inregistrare cu verificare prin email

Flux: Frontend POST /register-; Backend valideaza, salveaza in registration verifications, trimite cod SMTP -; Frontend POST /register/verify → INSERT users.

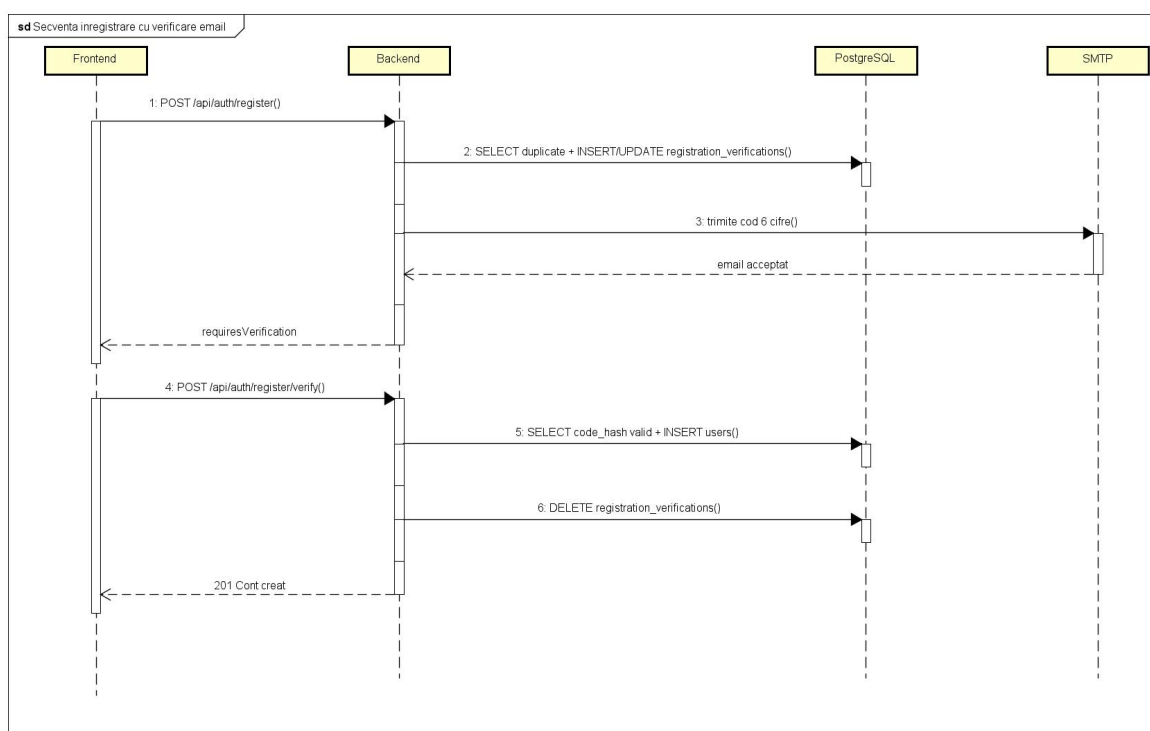


Figura 3.3: Diagrama de secventa - inregistrarea cu email

Instructiuni inserare: Participant,i: Utilizator, Frontend, Backend, PostgreSQL, SMTP. Mesaje: register, salveaza temp, trimite cod, verify, creare user. Notat,ii pe fiecare sageata.

3.4.3 Secventa pentru plata prin Stripe

Flux: Frontend POST create-checkout-session → redirect Stripe → success URL → confirmpayment → INSERT tickets, UPDATE locuri.

Instructiuni inserare: Participant,i: Utilizator, Frontend, Backend, Stripe, DB. Include metadata session si confirm-payment.

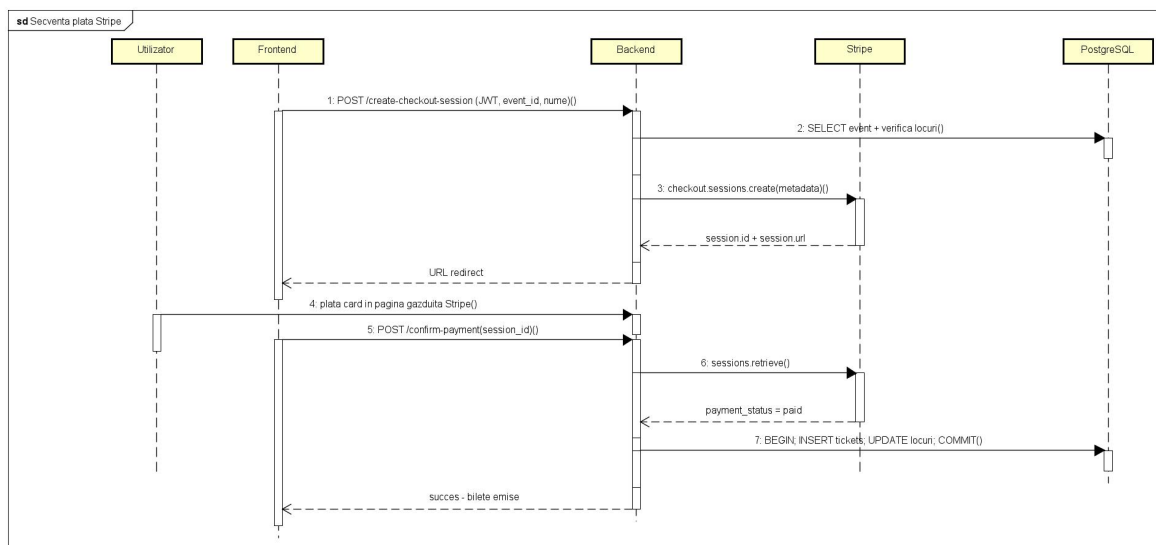


Figura 3.4: Diagrama de secventa - plata Stripe

3.4.4 Activitatea la check-in si admin

Instructiuni inserare: Flowchart: admin selecteaza eveniment → lista participanti → validare bilet → UPDATE check_in=true sau mesaj eroare

3.4.5 Proiectarea bazei de date

Tabele principale: users, events, tickets, password reset tokens, registration verification

Relatii: events 1:N tickets; users 1:N tickets (ON DELETE SET NULL); users 1:N token-uri reset; verificari inregistrare temporare pana la confirmare.

3.5 Proiectarea bazei de date

Tabele principale: user, events, tickets, password-reset-token, registration-verification

Relatii: events 1:N tickets; user 1:N tickets (ON DELETE SET NULL) user 1:N token-uri reset si verificari inregistrare temporare pana la confirmare.

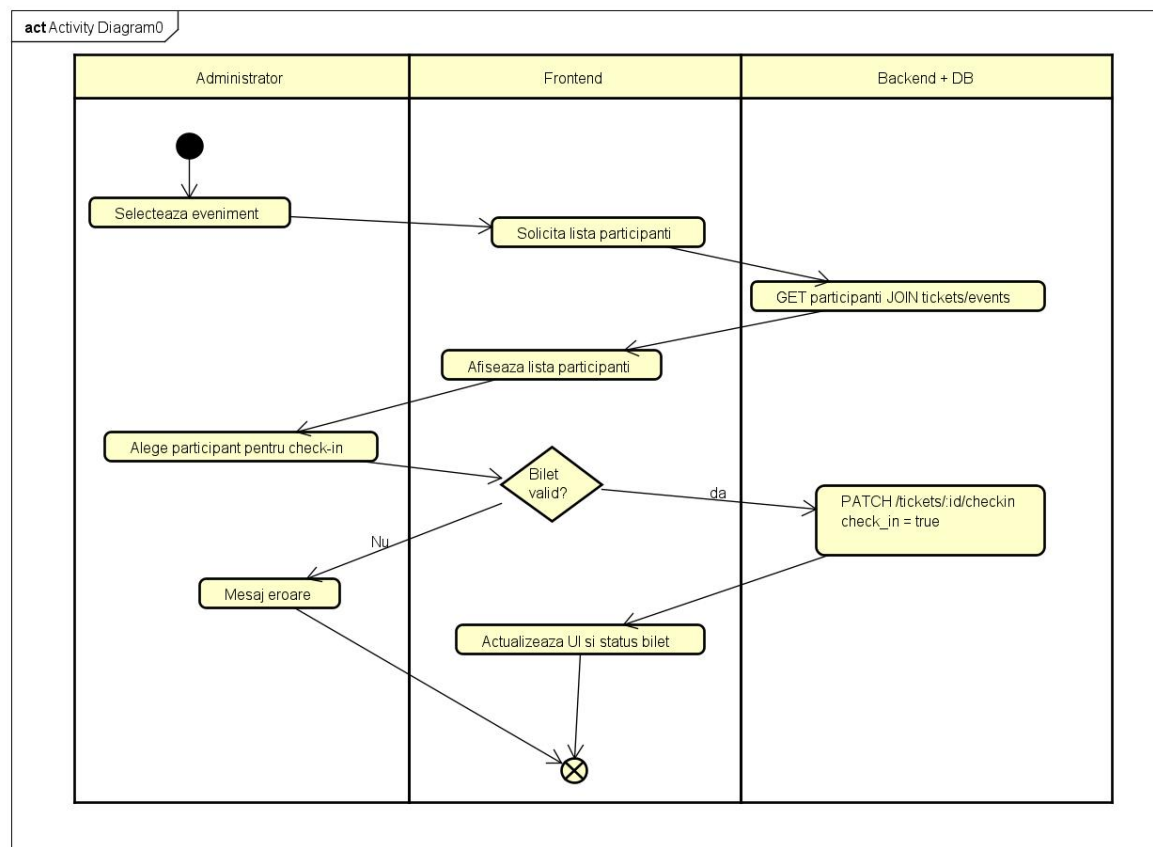


Figura 3.5: Diagrama de activitate - check-in

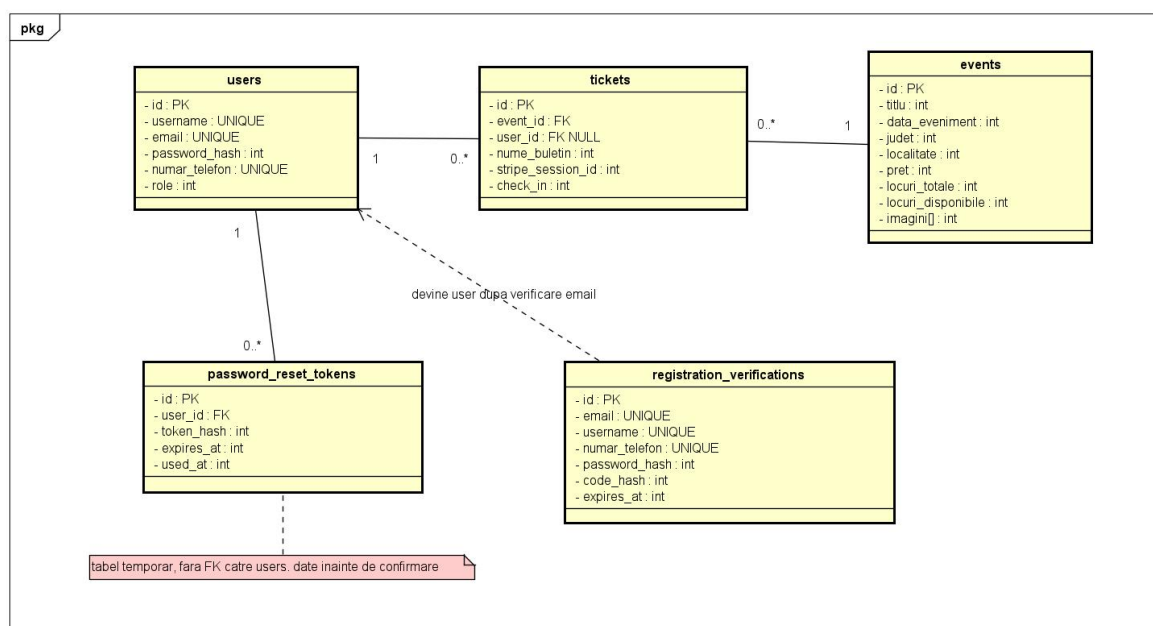


Figura 3.6: Diagrama entitate- relatie

3.6 Implementarea functionalitatilor cheie

3.6.1 Autentificarea si verificarea prin email

Generarea codului si a hash-ului

```
function genereazaCodVerificare() {  
    return String(crypto.randomInt(100000, 1000000));  
}  
function hashCodVerificare(cod) {  
    return crypto.createHash('sha256')  
        .update(String(cod).trim()).digest('hex');  
}
```

La POST /register/verify, codul este comparat cu code_hash, expirarea verificata, apoi utilizatorul inserat in users. Login-ul genereaza JWT:

```
const token = jwt.sign(  
  { id: user.id, role: user.role },  
  process.env.JWT_SECRET,  
  { expiresIn: '24h' }  
);
```

3.6.2 Plati prin Scribe

Crearea sesiunii Checkout (extras):

```
const session = await stripe.checkout.sessions.create({  
  payment_method_types: ['card'],  
  line_items: [{  
    price_data: {  
      currency: 'ron',  
      product_data: { name: `Bilet - ${event.titlu}` },  
      unit_amount: Math.round(Number(event.pret)  
        * 100),  
    },  
    quantity: cantitate  
  }],  
  mode: 'payment',  
  success_url: `${frontendUrl}/payment-success?session_id={CHECKOUT_SESSION_ID}`,  
  metadata: {  
    event_id,  
    user_id,  
    nume_participanti: JSON.stringify(nume)  
  }  
});
```

`confirm-payment` verifica `session.payment status === 'paid'`, insereaza cate un rand `tickets` per nume si decrementeaza `locuri_disponibile` intr-o tranzactie.

3.6.3 Gestionare evenimente

Crearea evenimentului foloseste `Multer` pentru imagini si validare judet, din lista fixa (42 judete). La `INSERT`, `locuri_disponibile = locuri totale`. Editarea permite adaugare incrementală de imagini. Stergerea elimina evenimentul si biletele asociate (CASCADE)

3.6.4 Frontend

`AuthContext` stocheaza `user` si `token` in `localStorage`. `apiFetch` atas_eaza `header Authorization`. Rute protejate: `/profile`, `/my-tickets`, `/admin` (`AdminRoute`).

3.7 Securitate

- ▷ Parole: `bcrypt` cost 10; niciodată în clar în DB.
- ▷ JWT: secret separat de Stripe; expirare 24h.
- ▷ Coduri email și token reset: SHA-256 în DB.
- ▷ Rate limit: max 3 coduri / 15 min (producție); cooldown 60s retrimiteri.
- ▷ CORS: doar `FRONTEND_URL`.
- ▷ Admin: ultimul admin protejat la ștergere/revocare.
- ▷ Ștergere cont: confirmare parolă obligatorie.

Implementarea respecta recomandările OWASP [3] pentru autentificare si gestionarea sesiunilor in contextul unei aplicatii web educationale/prototip production-ready.

3.8 Scenarii de utilizare pe tipuri de evenimente

Aceasta sectiune descrie modul in care aceleasi functionalitati ale platformei sunt valorificate in scenarii reprezentative. Scopul este de a demonstra flexibilitatea modelului simplu eveniment-bilet, fara a impune module separate per domeniu.

3.8.1 Scenariul 1: Conferința universitară

Context: Facultatea organizează o conferință științifică cu 150 de locuri, taxă de participare 120 RON, desfășurare în Constanța.

Pași organizator:

1. Administratorul creează evenimentul: titlu, descriere (program, speakeri), dată, județ Constanța, localitate, locație (sală), preț 120, locuri 150, durată 8 ore, imagini (banner conferință).
2. Publică link-ul către pagina principală; participanții filtrează după dată sau județ.
3. Monitorizează locuri disponibile și lista participanților.
4. La intrare, efectuează check-in pe baza numelui de pe bilet.

Pași organizator: Pași participant: înregistrare cont, verificare email, cumpărare bilet nominal, prezentare la receptie. Biletul din “Biletele mele” confirmă plata și identitatea.

3.8.2 Scenariul 2 în care concertul este în aer liber

Context: Asociație culturală organizează un concert local, 500 locuri, bilet 50 RON, galerie foto artiști.

Particularități: Galeria cu multiple imagini sporește atractivitatea paginii de detalii. Filtrarea pe județ, ajută publicul din zonă. Vânzarea poate include mai multe bilete într-o singură tranzacție Stripe (familie, grup de prieteni), fiecare nume completat corespunde unui bilet nominal.

Limitări actuale: Nu există categorii de preț, (VIP, standard) sau harta locurilor — extensii posibile prin adăugarea unui câmp `tip_bilet` în schema bazei de date.

3.8.3 Scenariul 3: Meeting corporate / training

Context: Firma de IT organizează un workshop de o zi, 30 locuri, participare gratuită sau cu preț simbolic

Context: Club sportiv local, turneu cu taxă de înscriere, necesitate nume conform buletinului. **Utilizare:** Chiar și la preț mic, Stripe permite monetizare. Alternativ, organizatorul poate seta preț 1 RON pentru testare. Check-in-ul din admin oferă lista de prezență pentru raport intern. Verificarea email la înregistrare reduce înscrierile fictive.

3.8.4 Scenariul 4: Competiție sportivă amatorială

Context: Club sportiv local, turneu cu taxa de înscriere, necesitate nume conform buletinului.

Utilizare: Câmpul nume buletin la cumpărare captează numele participantului. Check-in manual validează prezenta. Aceasta nu folosește aceleași mecanisme ca conferința, fără tabele suplimentare pentru echipe sau clasamente.

3.8.5 M

atribuția funcționalităților și scenariilor

Funcționalitate	Conf.	Concert	Meeting	Sport
Filtrare județ/dată	Da	Da	Da	Da
Galerie imagini	Da	Da	Opțional	Opțional
Plată Stripe RON	Da	Da	Da	Da
Bilete multiple / tranzacție	Da	Da	Rar	Da (echipe mici)
Check-in admin	Da	Da	Da	Da
Nume nominal (buletin)	Opțional	Opțional	Opțional	Da

Tabelul 3.2: Funcționalități utilizate în scenarii

3.8.6 Modelul unificat de date

Decizia de a nu specializa tabelele per tip de eveniment (conferință vs. concert) simplifică mentenanța și permite unui singur panou admin să gestioneze toate formatele. Câmpurile titlu, descriere, data eveniment, județ, pret, locuri totale și imagini sunt suficient de generice; semantica (“taxa conferință” vs. “bilet concert”) este pur textuală, definită de organizator.

Aceste scenarii pot fi reproduse în cadrul demonstrației aplicației și documentate prin capturi de ecran în capitolul 4.

3.9 Fragmente suplimentare de implementare

3.9.1 Rate limiting la înregistrare

Funcția `depasesteLimitaCoduri` limitează abuzul de retrimiteri cod email în producție:

```
function depasesteLimitaCoduri(row) {
  if (isDev) return null;

  const secundeDeLaUltimulCod =
    (Date.now() - new Date(row.last_code_sent_at).getTime())
    / 1000;

  if (secundeDeLaUltimulCod < COOLDOWN_RETRIMITERE_SEC) {
    return { status: 429, error: 'Asteapta inainte de
      un cod nou.' };
  }

  if (inFereastraDe15Minute(row.last_code_sent_at)
    && row.codes_sent >= MAX_CERERI_COD_15_MIN) {
    return { status: 429, error: 'Prea multe cereri
      de cod.' };
  }

  return null;
}
```

^In modul development (NODE_ENV diferit de 'production'), limitarea este dezactivata pentru a facilita testarea repetata fluxului de inregistrare fara a astepta cooldown-ul de 60 de secunde

3.9.2 Middleware autentificare JWT

Middleware-ul authenticateToken extrage token-ul din header Authorization: Bearer, verifica semnatura cu JWT SECRET si ataseaza req.user pentru rutele protejate:

```
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) return res.status(401).json({ error: 'Token
    lipsa' });

  jwt.verify(token, process.env.JWT_SECRET, (err, user) =>
    {
      if (err) return res.status(403).json({ error: '
        Token invalid' });
      req.user = user;
      next();
    });
}
```

requireAdmin verifica suplimentar req.user.role === 'admin', returnand 403 pentru utilizatori obisnui. Aceasta separare permite reutilizarea autentificarii pe rute mixte (ex. /auth/me vs. /events POST).

3.9.3 Validare evenimente la creare

La POST /events, backend-ul valideaza judetul din lista fixa, pretul pozitiv, locuri disponibile si proceseaza upload-ul Multer:

```
const JUDETE_VALIDE = ['Alba', 'Arad', ..., 'Vaslui', 'Vrancea'];

if (!JUDETE_VALIDE.includes(judet)) {
  return res.status(400).json({ error: 'Judet invalid' });
}

const locuriTotale = parseInt(locuri_totale, 10);

if (isNaN(locuriTotale) || locuriTotale < 1) {
  return res.status(400).json({ error: 'Locuri invalide' });
}
```

Imaginile sunt salvate in uploads/events/ cu nume unic (timestamp + random), iar URL-urile relative sunt stocate in coloana imagini de tip TEXT[].

3.9.4 Componenta AdminRoute (frontend)

Ruta /admin este protejata pe client prin verificarea roului din context:

```
function AdminRoute({ children }) {
  const { user, loading } = useAuth();

  if (loading) return <CircularProgress />;

  if (!user || user.role !== 'admin') {
    return <Navigate to="/login" replace />;
  }

  return children;
}
```

Protecția pe client este complementara, nu substitut pentru requireAdmin pe server — orice apel API admin fara token valid este respins indiferent de manipularea rutei React.

3.9.5 Gestionare erori si feedback utilizator

Frontend-ul centralizeaza apelurile API in apiFetch, interpretand raspunsurile JSON si afiseaza mesaje Material UI (Alert, Snackbar). Codurile HTTP 400/401/403/429/500 sunt mapate la texte in romana in componentele de formular.

Backend-ul returneaza structuri consistente error: string evitand expunerea

stack trace in productie. Logging-ul pe consola (`console.error`) asista debugging-ul in development.

Aceste fragmente ilustreaza detalii de implementare care completeaza diagramele si tabelele API, oferind cititorului o imagine concreta a codului sursa ManFast.

3.10 Modelul relational - detalii tavele

3.10.1 Tabel user

Stocheaza conturile utilizatorilor. Campuri: `id` (SERIAL PK), `username` (UNIQUE), `email` (UNIQUE), `password_hash`, `numar_telefon` (UNIQUE), `role` (`user|admin`), `created_at`. Rolul determina accesul la `/admin`.

Evenimentele publicate de administratori: `titlu`, `descriere`, `data eveniment`, `judet`, `localitate`, `locatie` (string compus pentru afisare), `pret`, `locuri totale`, `locuri disponib imagini` (TEXT[]), `durata ore`. La creare, `locuri disponibile` = `locuri totale`. Filtrarea pe home exclude evenimente expirate (`data + durata < NOW()`).

3.10.2 Tabele auxiliare

`registration_verifications` — stocare temporara inregistrarilor pana la cod email.
`password_reset_tokens` — token hash SHA-256, expirare 1h, used at la consum.

3.11 API REST - referinta

Prefix: `/api`. Tabelul 3.3 listeaza endpoint-urile de autentificare.

3.12 Configurare si deployment

3.12.1 Variabile de mdiau backend

DB *, JWT SECRET, STRIPE SECRET KEY, FRONTEND URL, SMTP *, MAIL FROM, ADMIN AUTHORIZATION PASSWORD. Frontend: VITE API URL.

Endpoint	Metodă	Descriere
/auth/register	POST	Trimite cod verificare email
/auth/register/verify	POST	Confirmă cod, creează cont
/auth/register/resend	POST	Retrimite cod
/auth/login	POST	Autentificare, returnează JWT
/auth/me	GET	Utilizator curent (JWT)
/auth/profile	PUT	Actualizare profil
/auth/password	PUT	Schimbare parolă
/auth/account	DELETE	Ștergere cont (parolă)
/auth/forgot-password	POST	Solicitare reset
/auth/reset-password	POST	Parolă nouă cu token
/auth/admin/create	POST	Promovare admin
/auth/admin/revoke	POST	Revocare admin

Tabelul 3.3: Endpoint-uri autentificare

Endpoint	Metodă	Descriere
/events	GET	Listă publică evenimente viitoare
/events/:id	GET	Detalii eveniment
/events	POST	Creare (admin)
/events/:id	PUT	Editare (admin)
/events/:id	DELETE	Ștergere (admin)
/events/admin/list	GET	Listă admin
/tickets/create-checkout-session	POST	Sesiune Stripe
/tickets/confirm-payment	POST	Confirmare plată
/tickets/my-tickets	GET	Biletele utilizatorului
/tickets/admin/participants	GET	Participanți (admin)

Tabelul 3.4: Endpoint-uri evenimente și bilete

3.12.2 Migrari baza de date

Scripturi npm: setup-db, migrate-judet-localitate, migrate-password-reset, migrate-email-verification, migrate-multi-tickets. Schema completa in database/schema.sql.

3.12.3 Flux instalare

1. **PostgreSQL:** CREATE DATABASE manfast
2. cd backend && npm install && npm run setup-db
3. Rulare migrări incrementale dacă proiect existent
4. npm run dev backend; cd frontend && npm run dev
5. **Promovare admin:** npm run promote-admin -- email@exemplu.com

3.12.4 Componente frontend

3.12.4.1 Rute React Router

/, /login, /register, /forgot-password, /reset-password, /events/:id, /profile, /my-tickets, /payment-success, /admin (AdminRoute).

3.12.4.2 Context autentificare

AuthContext expune user, token, login, logout, updateUser. **Token JWT în localStorage.** La apiFetch, header Authorization: Bearer.

3.12.4.3 Temă vizuală

darkTheme.js — paletă cyan pe fundal întunecat. Componente refolosibile: AuthCard, PageContainer, EventListItem, StatusChip.

3.12.5 Confirmare plată — pseudocod

```
BEGIN transaction
existing = SELECT tickets WHERE stripe_session_id
```

```
IF existing.count >= cantitate THEN COMMIT; RETURN dejaProcesat  
FOR nume IN numeDeInserat:  
  INSERT INTO tickets (event_id, user_id, nume_buletin, ...)  
  UPDATE events SET locuri_disponibile -= count  
COMMIT
```

Idemponteuța: reapelarea `confirm-payment` cu același `session_id` nu dublează biletele.

3.12.6 Testare manuală

Scenariu recomandat pentru demonstrație:

1. înregistrare + cod email;
2. login;
3. filtrare evenimente;
4. cumpărare bilet Stripe test 4242 . . . ;
5. verificare „Biletele mele“;
6. promovare admin;
7. publicare eveniment;
8. check-in;
9. editare profil;
10. ștergere cont test.

Aceste detalii completează documentația tehnică necesară pentru înțelegerea și reproductibilitatea soluției ManFast.

Capitolul 4

Prezentarea aplicației

Aplicația ManFast rulează în browser la `http://localhost:5173` (frontend) și comunică cu API-ul Express la portul 5000. În acest capitol sunt ilustrate interfețele principale.

4.1 Utilizarea aplicației — perspectiva utilizatorului

4.1.1 Pagina principală

Pagina principală afișează evenimentele viitoare sub formă de listă orizontală. Componenta `EventSearchFilters` permite filtrarea după nume, județ (select din 42 județe) și dată. Evenimentele expirate (data + durata) nu apar în listă.

Utilizatorul neautentificat poate naviga și filtra evenimente. Pentru cumpărare bilet este necesară autentificarea. Designul folosește carduri `EventListItem` cu titlu, locație, preț și locuri disponibile. Navbar-ul conține link-uri către „Evenimente“, „Biletele mele“ și „Profil“ (vizibile doar autentificat), plus buton „Panou Admin“ pentru rol admin.

Filtrarea pe județ utilizează lista completă județelor din `constants/judete.js`, aliniată la organizarea administrativă din România. Filtrul pe dată permite restrângerea rezultatelor la o zi specifică, util în planificarea participării la competiții.

4.1.2 Detalii eveniment

Pagina `/events/:id` prezintă titlul, galeria de imagini, descrierea, data, județul, localitatea, locația, prețul și locurile disponibile. Utilizatorul autentificat poate adăuga unul sau mai multe câmpuri „Nume participant“ și iniția plata Stripe.

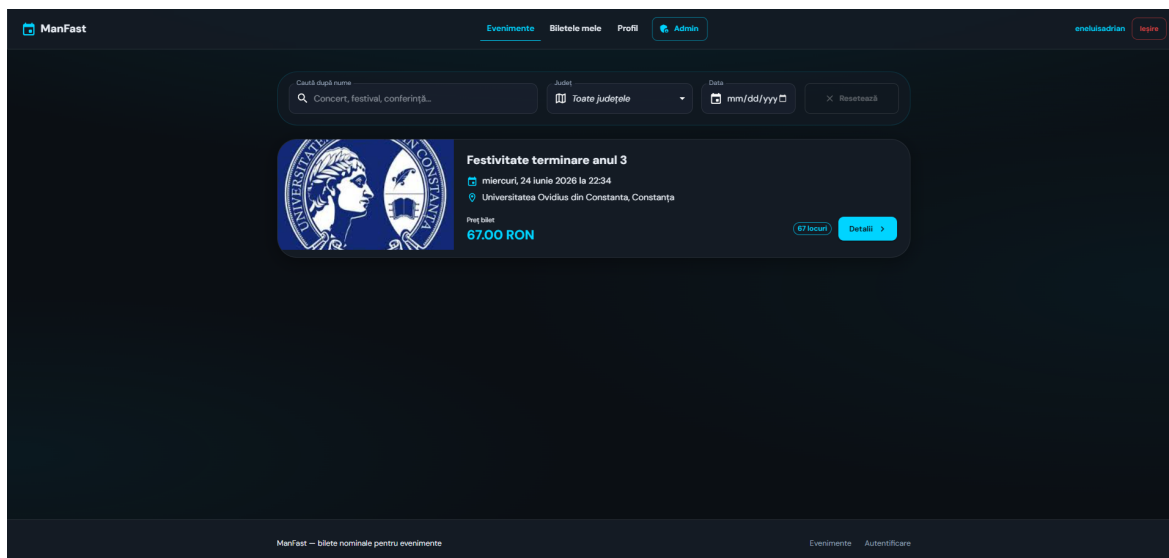


Figura 4.1: Pagina principală — listă evenimente și filtre

Galeria de imagini suportă mai multe fișiere încărcate de admin (max 8, 5MB/fișier). Layout-ul este responsive: pe desktop, detaliile și formularul de cumpărare pot apărea în coloane; pe mobil, stivă verticală. Validarea frontend verifică că toate numele sunt completate înainte de apelul API. La succes, utilizatorul este redirectat către Stripe Checkout hosted page — datele cardului nu tranzitează serverul ManFast.

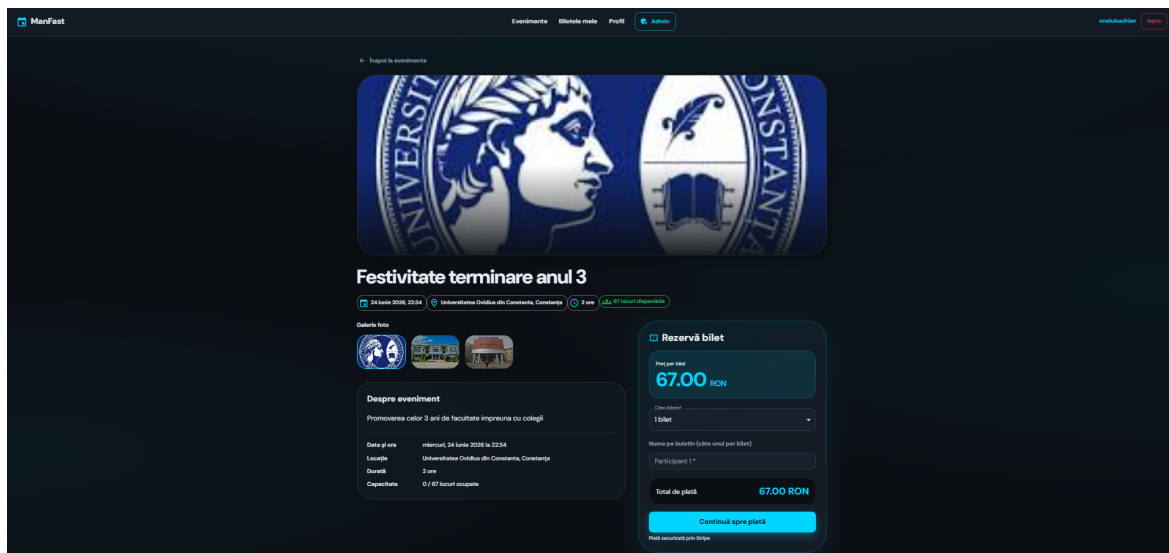


Figura 4.2: Pagina detalii eveniment și formular bilete

4.1.3 Înregistrare cont — pasul 1

Formularul de înregistrare colectează username, email, telefon, parolă. La submit se apelează `POST /api/auth/register` și se trece la ecranul de verificare.

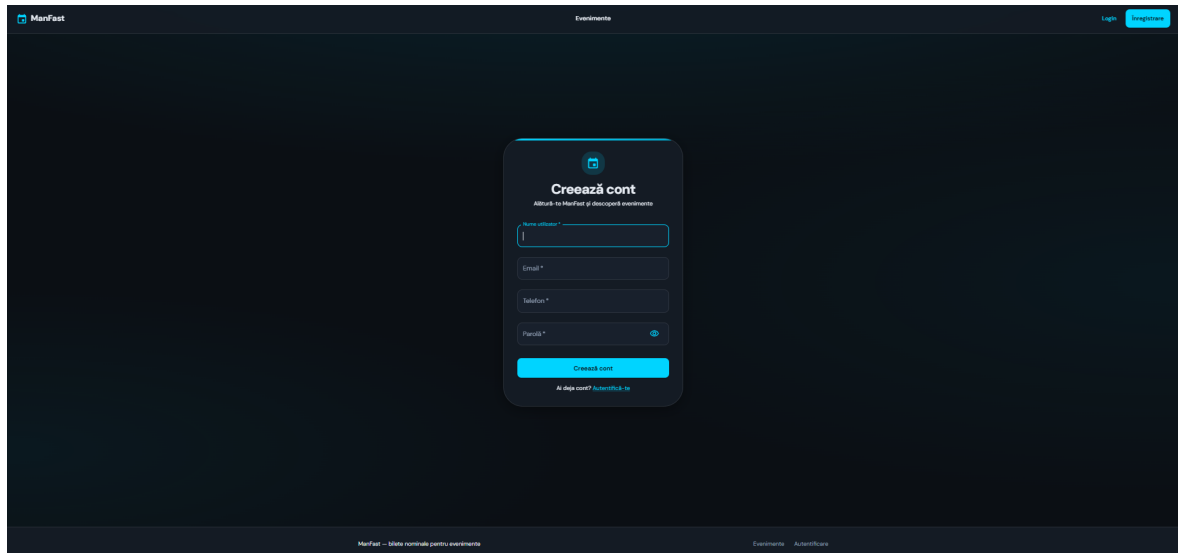


Figura 4.3: Formular înregistrare

4.1.4 Verificare email — pasul 2

Utilizatorul introduce codul de 6 cifre primit pe email. Opțiuni: retrimite cod, înapoi la formular.

4.1.5 Autentificare

Pagina `/login` permite autentificarea. Link-uri către „Am uitat parola“ și „Înregistrează-te“ (transfer email/parolă prin state).

4.1.6 Confirmare plată

După Stripe Checkout, utilizatorul revine la `/payment-success?session_id=...`. Frontend confirmă plata și redirecționează spre bilete.

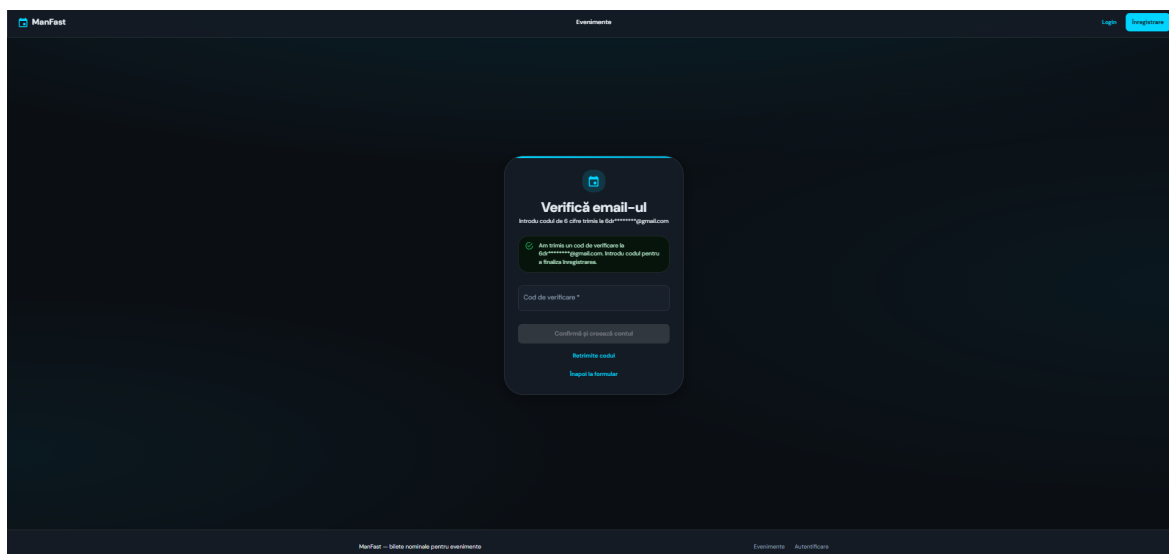


Figura 4.4: Verificare email cu cod

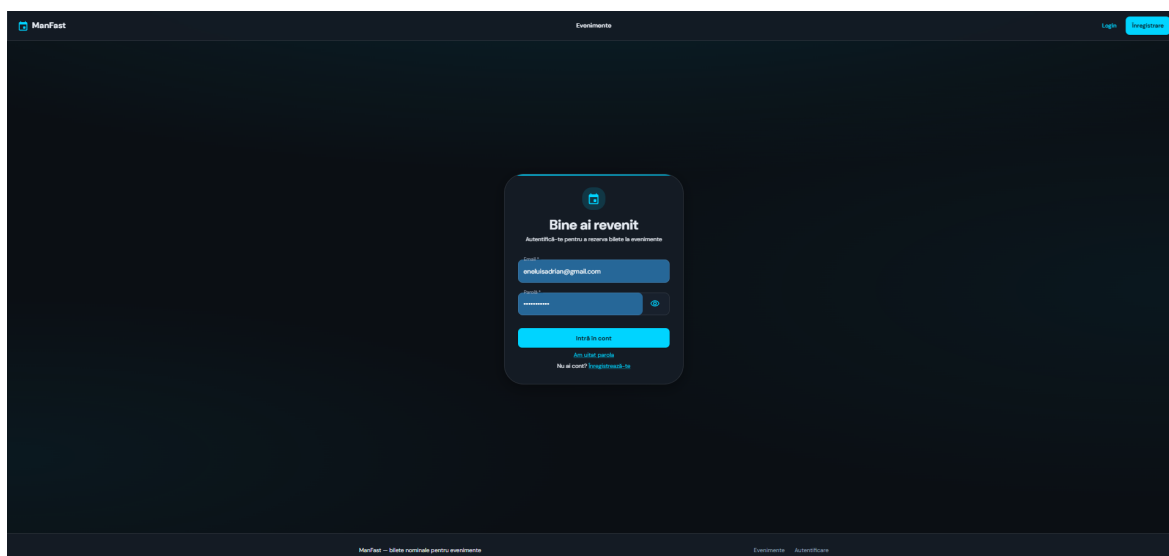


Figura 4.5: Pagina login

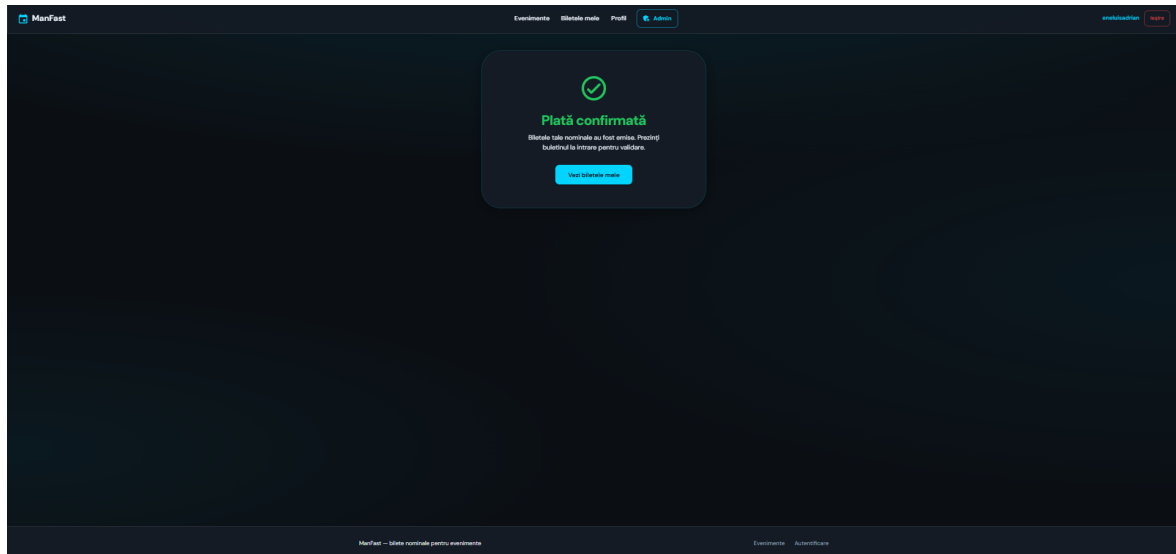


Figura 4.6: Pagina succes plata

4.1.7 Biletele mele

`/my-tickets` grupează biletele în „viitoare“ și „trecute“. Fiecare card arată evenimentul, numele de pe bilet, status (Plătit / Check-in / Expirat).

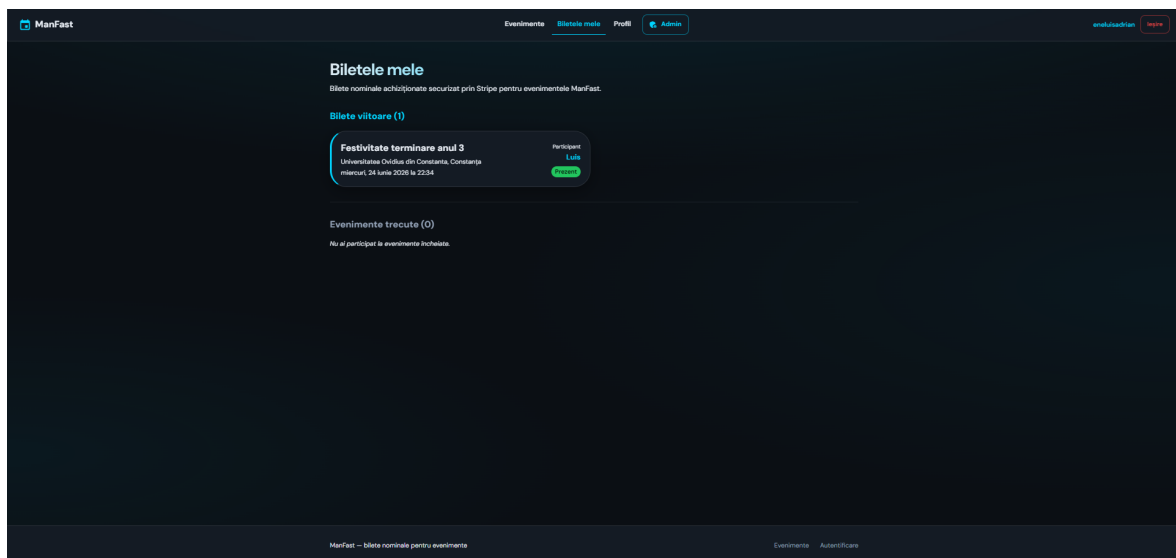


Figura 4.7: Biletele mele

4.1.8 Profil utilizator

`/profile`: mod vizualizare și mod editare; schimbare parolă; zonă roșie ștergere cont.

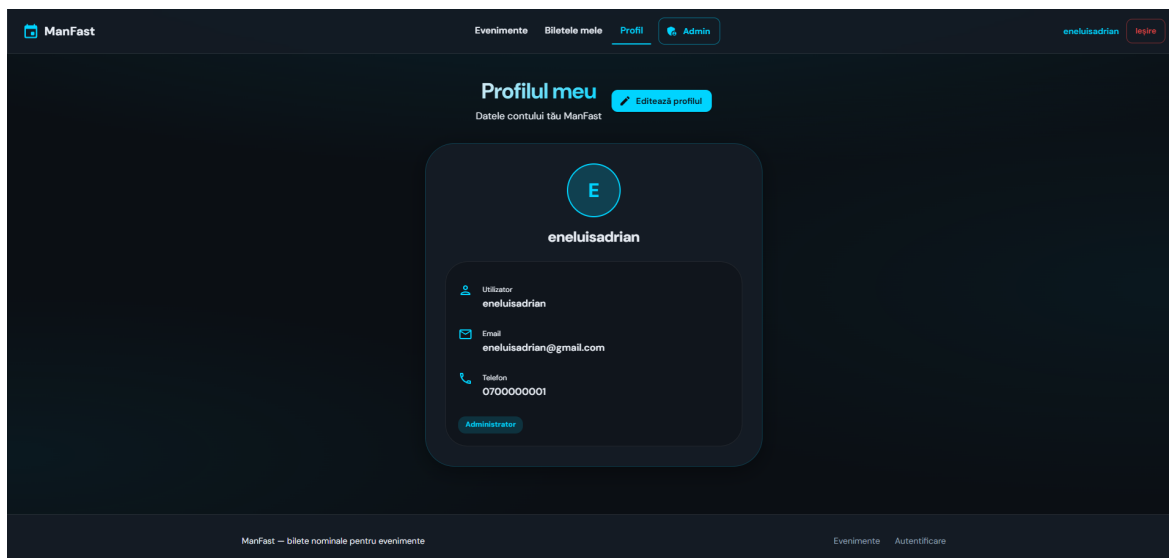


Figura 4.8: Profil utilizator

4.1.9 Resetare parolă — pasul 1 (opțional)

Formularul „Am uitat parola“ acceptă email sau telefon; backend-ul trimite link de reset pe email (mesaj generic pentru confidențialitate).

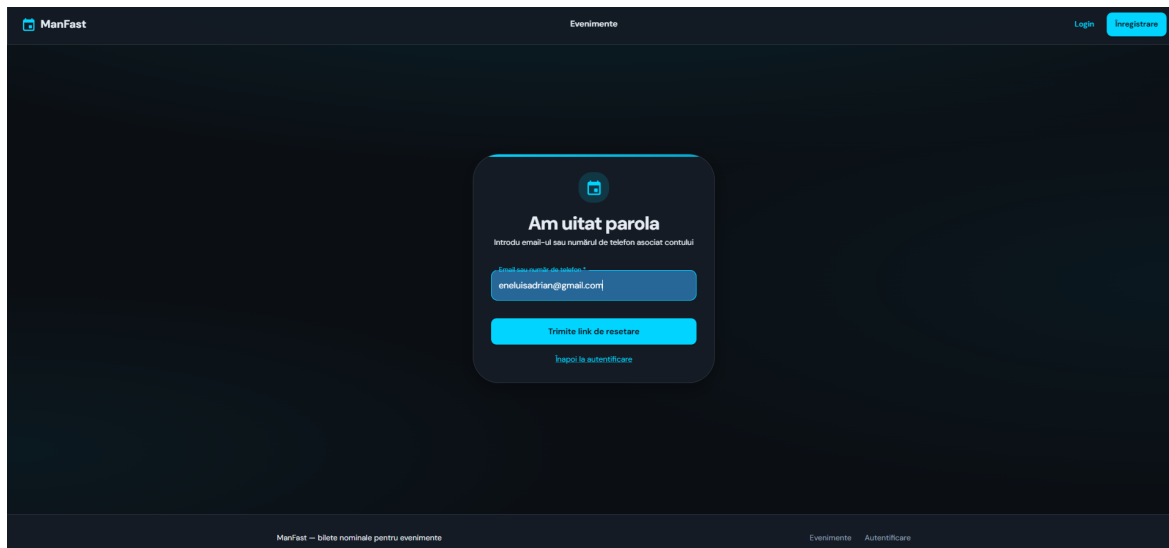


Figura 4.9: Pagina Am uitat parola (opțional)

4.1.10 Resetare parolă — pasul 2 (opțional)

După solicitarea resetării, utilizatorul accesează link-ul din email (`/reset-password?token=...`) și setează parola nouă. Token-ul este validat server-side (hash SHA-256, expirare 1h).

Figura 4.10: Formular parola noua (reset)

4.2 Experiență utilizatorului și design responsive

4.2.1 Parcursul utilizatorului (user journey)

Fluxul complet al unui participant nou poate fi rezumat astfel: descoperire eveniment pe Home → consultare detalii → redirect la Login/Register dacă nu este autentificat → înregistrare cu verificare email → revenire la eveniment → completare nume participanți → plată Stripe → confirmare → vizualizare bilet in Biletele mele → prezentare la check-in (validat de admin)

Fiecare pas este marcat de feedback vizual: stări de încărcare (`CircularProgress`), mesaje de eroare contextualizate, redirect automat după acțiuni reușite. Transferul email/parolă de la Login la Register (via `location.state`) reduce fricțiunea pentru utilizatorii care încep de pe pagina greșită.

4.2.2 Adaptare mobil și accesibilitate

Material UI 9 oferă componente responsive (`Grid`, `Stack`, `useMediaQuery`). Navbar-ul comută la meniul hamburger pe ecrane înguste; filtrele pe Home se stivează vertical. Formularele de autentificare sunt centrate în `AuthCard` cu lățime maximă, ușor de utilizat pe telefon.

Au fost remediate probleme de accesibilitate: focus corect după închiderea meniului mobil (evitare `aria-hidden` pe elemente focusabile), utilizarea etichetelor `label` pe câmpuri, contrast ridicat în tema dark (cyan pe fundal întunecat).

4.2.3 Mesaje de eroare și stări goale

Listele goale (fără evenimente, fără bilete) afișează text explicativ și sugestii (ex. „Modifică filtrele” sau „Cumpără primul bilet”). Erorile API (429 rate limit, 401 sesiune expirată) sunt traduse în mesaje inteligibile, fără detalii tehnice expuse utilizatorului final.

4.3 Utilizarea aplicației — perspectiva administratorului

Administratorul accesează `/admin` după autentificare cu rol `admin`. Componenta `AdminRoute` verifică token JWT și rol; utilizatorii obișnuiți sunt redirectionați.

Panoul centralizează gestionarea evenimentelor și a participanților. Administratorul poate crea evenimente noi, edita pe cele existente (inclusiv adăugare imagini incremental), șterge evenimente (cu confirmare dialog) și vizualiza participanți cu status check-in. Gestionarea administratorilor se face într-un dialog separat, protejat de parolă de autorizare configurată în `.env`.

4.3.1 Panou admin — dashboard

Listă evenimente cu acțiuni editare/ștergere; tab participanți; check-in.

4.3.2 Creare / editare eveniment

Dialog cu câmpuri: titlu, descriere, dată, județ, localitate, preț, locuri, durată, upload imagini.

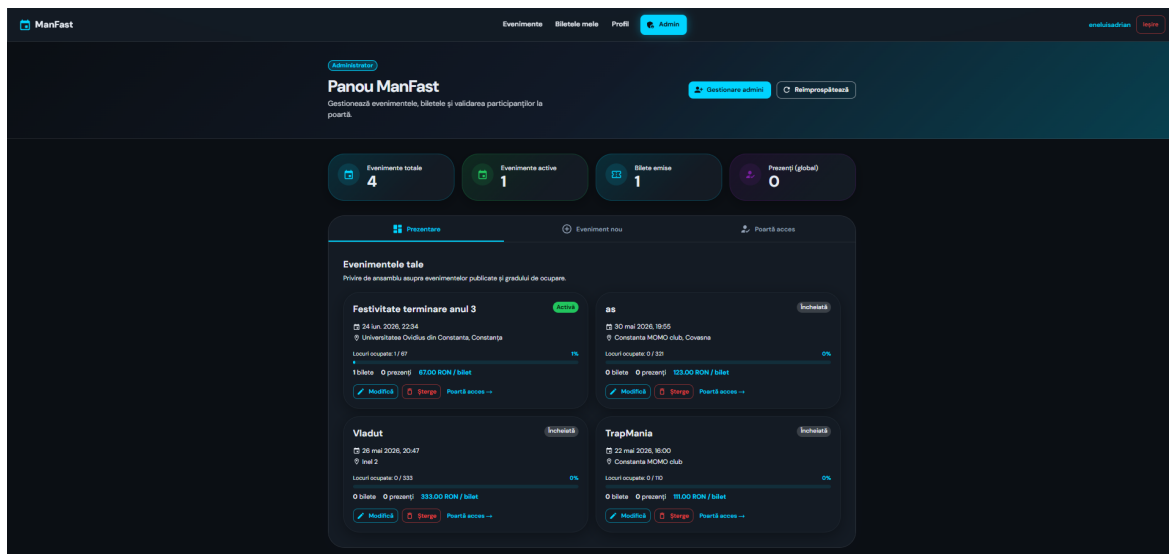


Figura 4.11: Panou admin — dashboard

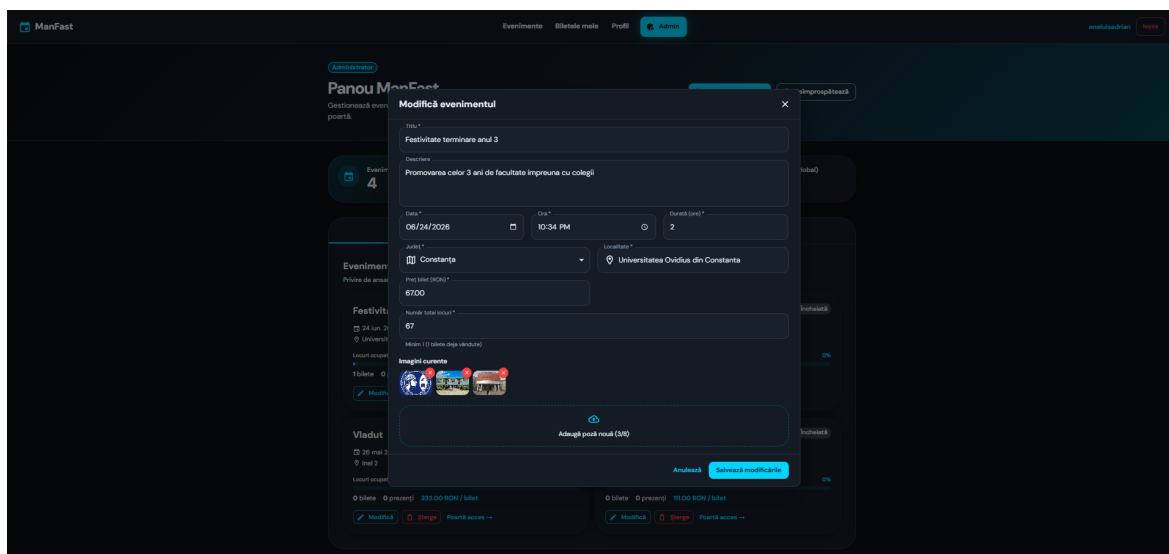


Figura 4.12: Dialog editare eveniment

4.3.3 Check-in participant

Marcarea unui participant ca prezent (buton check-in sau toggle)

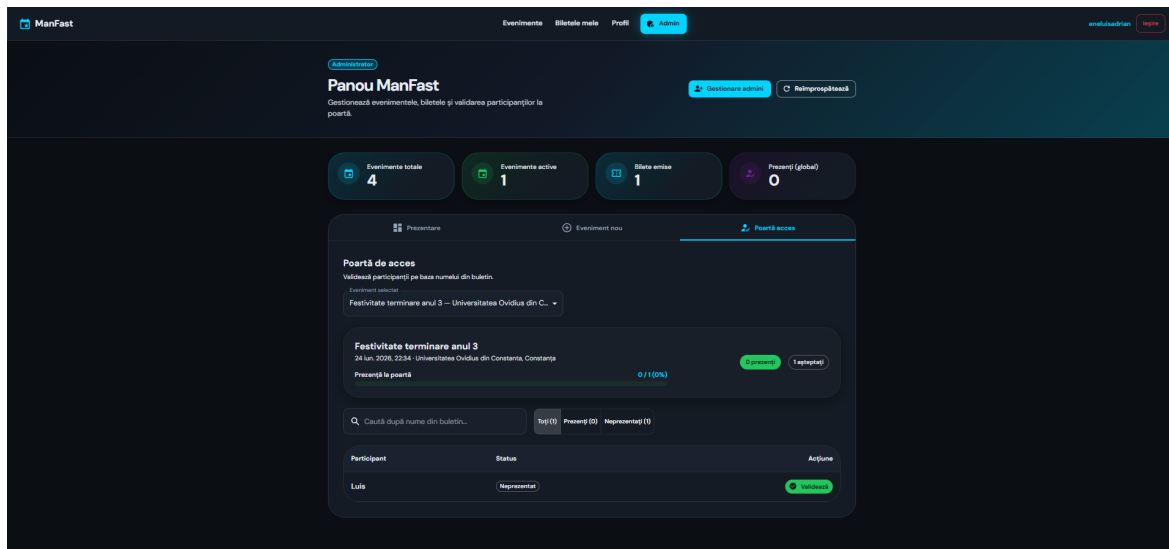


Figura 4.13: Check-in participant

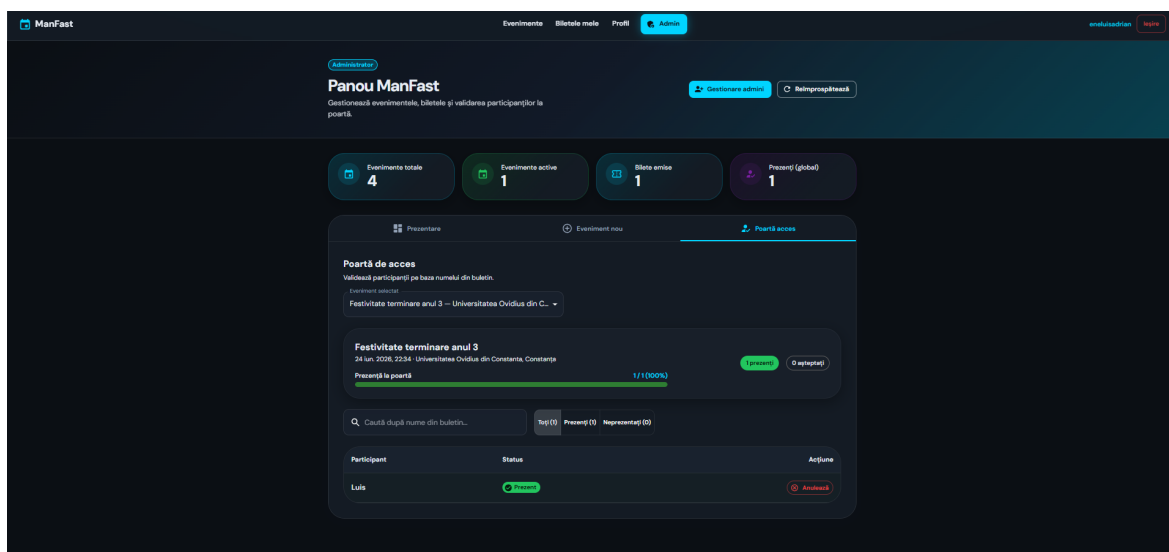


Figura 4.14: Check-in participant validat

4.3.4 Gestionare administratori

Dialog promovare sau revocare admin cu parola autorizare.

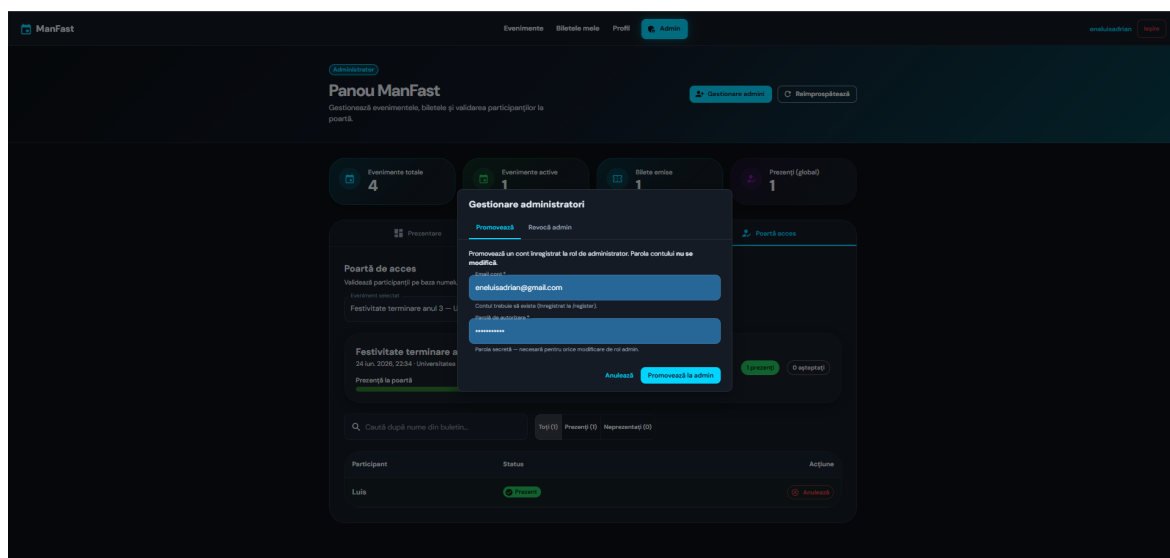


Figura 4.15: Gestionare administratori

4.3.5 Conferință sau simpozion

Pagina de detalii poate include în descriere: programul pe zile, lista speakerilor, link-uri către site-ul conferinței. Imaginile încărcate pot fi logo-ul evenimentului și fotografii din ediții anterioare. Filtrele de pe Home permit găsirea conferințelor dintr-un anumit județ sau dată — util pentru participanți care caută evenimente academice regionale.

Formularul de cumpărare bilet colectează numele participantului (pentru ecuson sau listă oficială). După plată, „Biletele mele” afișează evenimentul ca „viitor” până la data desfășurării; ulterior trece automat în „trecute”.

4.3.6 Concert sau spectacol

Organizatorul pune accent pe elementele vizuale: mai multe imagini (artiști, poster), descriere atractivă, preț și locuri limitate pentru a crea urgență. Utilizatorii pot cumpăra bilete pentru mai multe persoane introducând câte un nume per câmp — potrivit pentru ieșirile în grup.

Navbar-ul și tema dark oferă un aspect modern, aliniat la așteptările publicului tânăr pentru evenimente culturale. Pagina de succes după Stripe confirmă achiziția și redirecționează spre bilete.

4.3.7 Meeting, workshop sau seminar

Pentru evenimente profesionale cu număr mic de locuri, administratorul poate urmări în timp real câte locuri au rămas disponibile. Check-in-ul la intrare în sala de training confirmă prezența pentru certificat sau raport HR. Resetarea parolei și editarea profilului permit participanților să-și mențină contul pe termen lung, pentru evenimente recurente organizate de aceeași entitate.

4.3.8 Eveniment sportiv (caz secundar)

La competiții, numele de pe bilet poate corespunde actului de identitate al sportivului. Admin-ul bifează check-in la înregistrarea la concurs. Interfața nu afișează distincții specifice (categorie greutate, gen), dar fluxul tehnic este identic — organizatorul completează manual detaliile în titlu și descriere.

4.3.9 Recomandări pentru capturi de ecran (capitol 4)

Pentru o documentație reprezentativă, autorul poate crea în baza de date trei evenimente demonstrative:

- ▷ „Conferința Națională Informatică 2026“ — accent pe descriere lungă și dată.
- ▷ „Concert Jazz Constanța“ — accent pe galerie imagini.
- ▷ „Workshop React pentru Începători“ — accent pe locuri limitate (ex. 25).

Capturile din figurile 4.1–4.8 pot folosi oricare dintre aceste evenimente; important este ca textul lucrării să reflecte diversitatea cazurilor, nu doar un singur domeniu.

4.3.10 Accesibilitate și public larg

Indiferent de categorie, formularele respectă etichetare clară, mesaje de eroare în română și layout responsive. Participanții mai puțin obișnuiți cu tehnologia (ex. invitați la conferință) beneficiază de flux liniar: Home → Detalii → Login/Register → Plată → Bilete.

Aceste exemple completează descrierea interfeței și leagă capturile de ecran de cazuri reale de utilizare în domeniul conferințelor, concertelor și meeting-urilor.

Capitolul 5

Concluzii

5.1 Rezultate obținute

Lucrarea de licență a avut ca obiectiv dezvoltarea **aplicației web pentru organizarea și gestionarea evenimentelor**, orientate întâi spre conferințe, meeting-uri, concerte și evenimente culturale, cu suport complementar pentru competiții sportive. Obiectivele propuse au fost atinse:

- ▷ Aplicație full-stack funcțională: React + Express + PostgreSQL.
- ▷ Autentificare completă: înregistrare cu verificare email, login JWT, resetare parolă, profil, ștergere cont.
- ▷ Listare și filtrare evenimente (nume, județ, dată).
- ▷ Plăți Stripe Checkout în RON; bilete nominale și „Biletele mele”.
- ▷ Panou admin: CRUD evenimente cu imagini, check-in, gestionare admini.
- ▷ Documentare: arhitectură, ERD, UML, API REST, scenarii pe tipuri de evenimente.

Soluția demonstrează integrarea unor tehnologii moderne într-un produs coerent, adaptat contextului românesc (județe, RON, email Brevo).

5.2 Limitări

- ▷ Lipsă teste automate (unit/integration) și pipeline CI/CD.
- ▷ Promovarea primului administrator necesită script SQL sau `promote-admin`.
- ▷ Emailurile depind de configurarea SMTP; în development, codurile pot apărea în consola backend.
- ▷ Confirmarea plății se bazează pe apel frontend `confirm-payment`; webhook Stripe ar îmbunătăți robustețea.
- ▷ Un singur tip de preț per eveniment — fără categorii VIP/standard sau locuri numerotate.

5.3 Dezvoltări viitoare

Dezvoltarea aplicației poate fi continuată prin abordarea limitărilor identificate și adăugarea de noi funcționalități:

- ▷ Categoriile de bilete (conferință: early bird / regular; concert: VIP).
- ▷ Export participanți CSV din panoul admin.
- ▷ Notificări email la cumpărare bilet reușită.
- ▷ Paginare evenimente și caching.
- ▷ Webhook Stripe pentru confirmare server-side.
- ▷ Teste automate și deploy containerizat (Docker).
- ▷ Aplicație mobilă sau PWA pentru check-in offline.
- ▷ Module opționale pentru evenimente sportive (echipe, program).

Aplicația reprezintă o bază solidă pentru extindere într-un produs destinat organizatorilor de conferințe, concerte și meeting-uri, cu accent pe simplitate, securitate și experiența utilizatorului.

Referințe bibliografice

- [1] R. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000.
- [2] Stripe Inc. Stripe checkout documentation. <https://stripe.com/docs>, 2024. Accesat 2026.
- [3] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [4] OWASP Foundation. Authentication cheat sheet. <https://cheatsheetseries.owasp.org/>, 2024. Accesat 2026.
- [5] M. Jones, J. Bradley, and N. Sakimura. RFC 7519 – json web token (jwt). Technical report, IETF, 2015.
- [6] N. Provos and D. Mazières. A future-adaptable password scheme. In *USENIX Annual Technical Conference*, 1999.
- [7] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2024. Accesat 2026.
- [8] Meta Platforms. React documentation. <https://react.dev/>, 2024. Accesat 2026.
- [9] MUI Team. Material ui documentation. <https://mui.com/>, 2024. Accesat 2026.
- [10] Eventbrite Inc. Eventbrite platform. <https://www.eventbrite.com/>, 2024. Accesat 2026.